

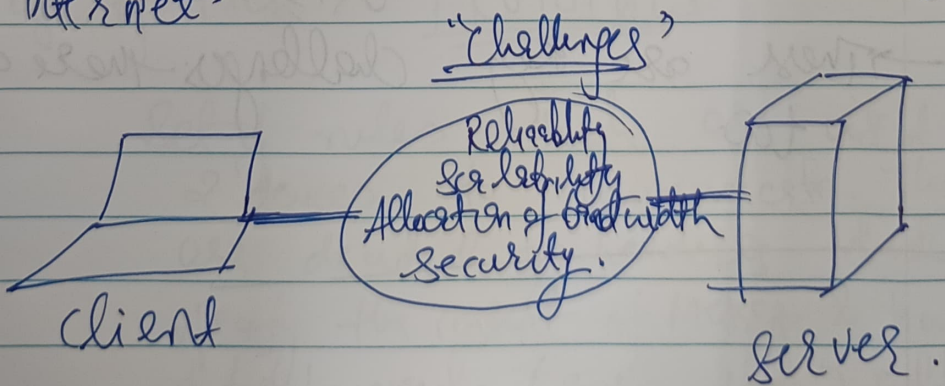
COMPUTER NETWORKS

A CN is formed by 2 or more devices connected together. They might be connected to share data, to do computations together or to share resources.

Ex A network where 2 computers & a printer is connected & both want to give command to the printer. So resource sharing happens due to computer networks.

Ex In home, def, Alex, etc. connected over networks. In office devices connected over router.

We focus on INTERNET: the largest network
50% of world's popⁿ is connected through internet.



how can we browse a website in some other corner of world?

① Reliability (how to ensure that our request reaches the server reliably? And data that we receive from server reaches us reliably?)

② Scalability (How to ensure all devices are connected & they get all services, no address collisions.)

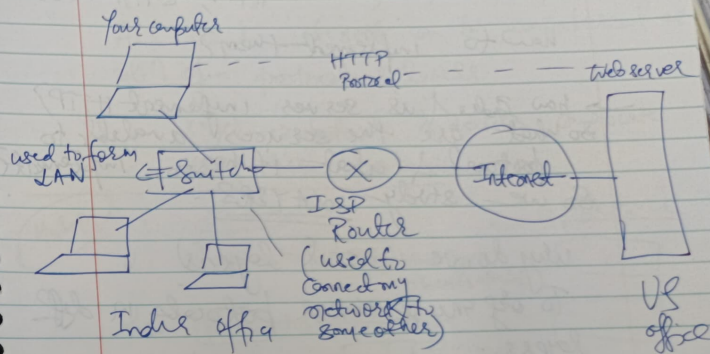
③ Allocation of Bandwidth

(how to ensure that there is no subnetwork that consumes all bandwidth of internet)

④ Security (how to ensure data reaches server securely & we get it back securely?)

→ These are key challenges there are more too

Basics of CN



RTS: Connection establishment, finding a path to web server, talking to it, deciding the format of the request, response, processing the format, making efficient use of bandwidth, adding new devices to internet. How to manage all of these? PROTOCOLS

↓
Set of rules which are used by 2 devices to communicate. They are decided by headers which specify the content of message & how it is to be processed. (Source, destination address)

Type of protocols: IP, TCP, UDP,
HTTP, SMTP, ...

how to implement them?

→ how can I as server implement HTTP?
So what are the services available to
protocol & what it has to implement?
So we study LAYERS.

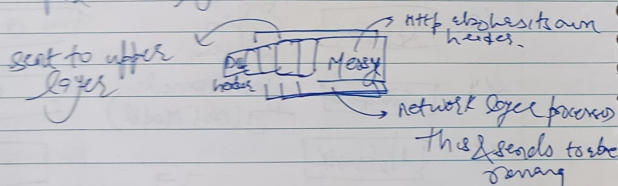
why do we need layers?

To organize these protocols in diff
layers.

when we say HTTP is an applⁿ layer protocol
means it doesn't have to implement
anything ~~provided~~ by lower / transport
layer protocol. It may assume all those
services are available. It doesn't have
to worry about ethernet too. WiFi/
wired Kr bhi no tension. I just need
to use the services & implement my own
layer services.

- sender every on app, say web browser
lower layers attach their own
headers once previous. So ~~data~~ msg
w/ data link layer headers is sent over
the physical medium which passes thru
data to the upper layers.
- data link layer on receiver only tells
to DL protocols like 802.11
802.3
- It only understands the protocols in that
specific layer.

upto DL layer → 5 headers.



Data link → Network → to server port → Application

Receiver after all processing occurs http
message only. They don't worry whether
these was ethernet/wifi in other
layers. They just talk to each other using
http. say if transport layers are talking
to each other in TCP, they do it & don't worry
abt underlying network.

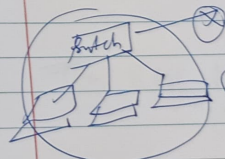
So we get the adv of ENCAPSULATION
messages get encapsulated further &
forwarded correspondingly.

* OSI Model (7 layers) (theoretical & never
accepted in implementation)

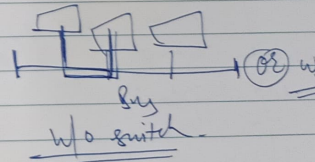
Most frequent used → TCP/IP Model
(4 layers)

OSI →

- ① Physical layer → wireless/fiber optic/copper wire
etc. All about wires & signals.
- ② Data link layer → uses physical medium to
comm. This is mainly for
one-to-one connections.



Communication in
local area network
(LAN)



happens using data link d
through switch / Ethernet
(for header processing)

In DLL, mostly these things happen

There can be errors. Devices might be at diff speeds so flow control has to be handled. In ethernet (shared medium) access to the shared med must be controlled by DLL for collision handling. So error

Control, Error detect, flow control

~~control~~

③ Network Layer

12/8/26

CN class 2

Now do packet delay & loss occurs?

packet loss $\Rightarrow$ drop

when in queue $\Rightarrow$ delay. "Queue delay" ①

"Processing delay" ②

{ Packet changes }

"Transmission delay" ③

{ putting bits on wire }

(analogy: toll booth)

"Propagation delay" ④

{ time to get from src
↓
dest }
bits are on wire.

$\rightarrow$ T.R. vs Bandwidth

$$C = B \log_2 (1 + S/N)$$

$y \rightarrow 0, c \rightarrow 0$

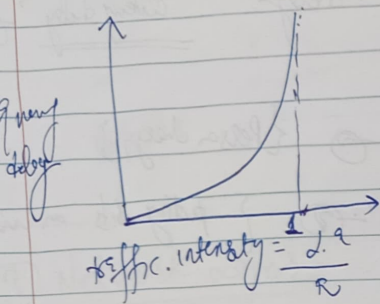
(more is the more
loss is up)

more B is for diff medium

$\rightarrow$ store & fwd.

very $\Rightarrow$
 $\text{Arrival Rate} + \text{packet size} < \text{transmission rate}$

$\downarrow A > R \Rightarrow \text{queue formation}$



① which packet to drop?

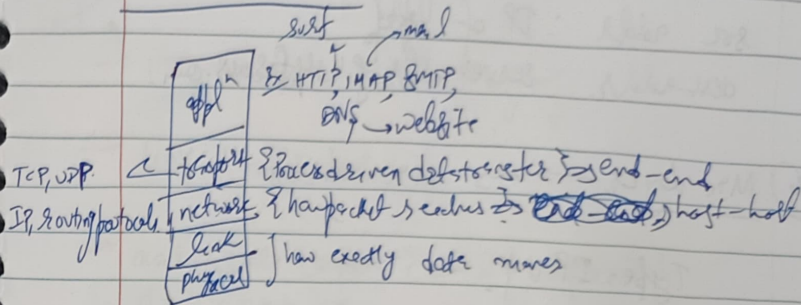
(force route) hop value.
 (command)

$\rightarrow$ virtual division

Network Design

Types of attacks

TCP/IP stack



class 3
 17/8/26 $\rightarrow$ http forever. com
 $\rightarrow$ wireless.

(refresh $\rightarrow$ request.
 http response)

hyper text transfer protocol.

methods: get, put, post, delete

① http/1.1

(use this protocol version.)

conn: keep-alive.
 user-agent: Mac.

$\rightarrow$ http uses
 TCP as its
 protocol

②

Trans. port layer: src

default: 80.
 dest port.

www. fl. com; do
 also same.

* Packet capture.

⑤ Network Layer : Device $\rightarrow$ Device.

src addr : IP of laptop
dest addr. server addr of http://www.com.

⑥ Mac layer $\rightarrow$ has mac address

Type: IP v4.

Ch-2

Application layer

$\rightarrow$ hide complexity w/in applications

$\rightarrow$ public vs private addr. { server must have public addr }
peer-peer architecture

$\rightarrow$ difficult today so client-server preferred.

TCP network conn. vs UDP

$\rightarrow$ Socket vs Port

RFC: request for comments

$\rightarrow$ reliability & speed, time

clear

19/02/26

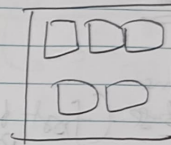
$\rightarrow$ HTTP $\rightarrow$ uses TCP $\rightarrow$ data integrity $\checkmark$

$\rightarrow$ 404 (page not found)

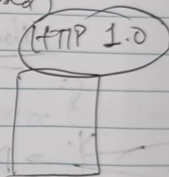
$\rightarrow$ mainly stateless. { scheduled refresh }
ATM is stateless

$\rightarrow$ best effort service. (best effort)

or



server
(images or web page)

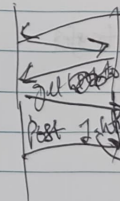


client

design http protocol.

$\rightarrow$ Client needs to know & find path to server $\leftrightarrow$

TCP vs
bits)



{ TCP conn is always end-end. No wire is reserved }
 $\rightarrow$ after Post, conn closed

* on refresh, which to be sent $\rightarrow 1+2+3+4+5$

(T_{RTT} → RTT negs)

→ For multiple object transfer w/ HTTP 1.0 →

path for each src → obj.

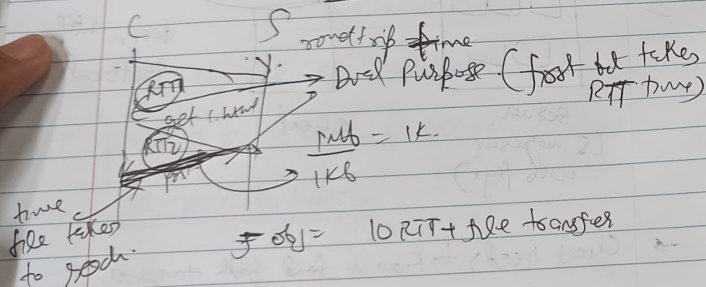
(1)

by default, test in response.

* can't open + close → # channels

to send all requests in one connection → keep it open until everything arrives.

Non-persistent



For persistent → 2 RTT + file transfer

Persistent

* unique identifier by server
(Cookie / MAC / MSN)

→ that is why
v2. ind

2006/5
24/9/06

Cookie system

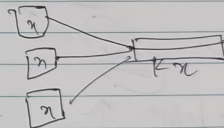
HTTP cookies, what protocol doesn't provide, it will allow third party cookie.

Amazon → collected info → send info ahead

→ EUDR - GDPR

Web caches / proxy server

→ ? solution for both client parties



Time in Networking

- ① Processing (Router needs headers)
- ② Queuing - depends on traffic
- ③ Transmission: L/R bits/sec
- ④ Propagation: d/s m/s

dist d median $d/2$ Prop $\uparrow$ $\uparrow$ transmission L

$$\frac{10000}{10^6} = 10^{-3} = 1ms$$

Packet size does not affect $\uparrow$ 4 Mbps, $x = 1000$ bits, $d = 2000$ km

$$d/s = \frac{L}{R} = \frac{1000}{10^6} \text{ (seconds)}$$

$$d/s = \frac{2000}{10^6} = 10^{-3} \text{ (seconds)}$$

$$\frac{2 \times 10^6}{2 \times 10^8} = 10^{-2} = 10ms \text{ (10)}$$

$$trans = \frac{10^5}{10^8} = 10^{-3} s = 1ms$$

$$prop = \frac{2 \times 10^6}{10^8} = 10^{-2} \text{ sec} = 10ms$$

Small packets + long dist $\rightarrow$ prop $\uparrow$
 Big packets + small dist $\rightarrow$ trans $\uparrow$

HTTP resp time = $dRTT + \text{transmission time}$

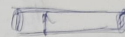
prop $\rightarrow$ bits sent at rate

In-state > out state $\rightarrow$ queue

traffic intensity = $\frac{L}{R} \times \text{packet/sec}$

Packet size $\rightarrow$ link port

link capacity $\rightarrow$ prop.



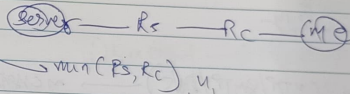
$\rho < 1$ - no delay
 $\rho = 1$ - exponential growth
 $\rho > 1$ - packet drop queue

delay: $\frac{1000 \times 1000}{2 \times 10^6} = 0.5$ sec

throughput: per second

0 - 0.8: queue light
 0.8 - 0.95: delay $\uparrow$
 0.95 - 1: exponential growth
 > 1 : queue, packet drop

Throughput: per second bits $\frac{1}{s}$



[case link] $\rightarrow v_2$

For user, state = R/N

$\min(R_s, R_c, R/N) = 2$

Packet loss: Congestion security, network

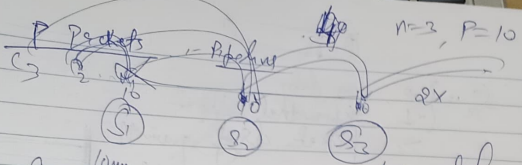
$3 \times \frac{10000}{10^6} = 3 \times 10^{-2} = 3\%$

TCP: reliable UDP: unreliable

Smack full: to send feedback checker status

$\frac{L}{R} \rightarrow \frac{1}{s}$

$N \times \frac{1}{s}$



$$(N) \frac{L}{R} + P \left(\frac{L}{R} \right)$$

Mohamed.

TCP/IP 5 layer network model.

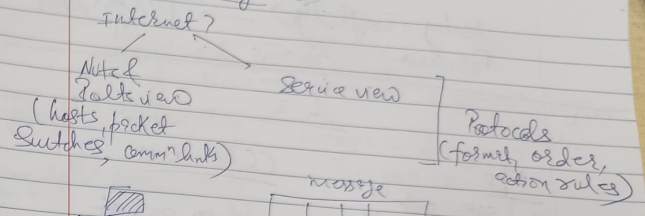
$$4 \times \frac{10000}{10^6} \rightarrow 3 \times \frac{10000}{10^6}$$

Logan (system in feet)
Airline analogy

App - Application (HTTP/SMTP/DNS) → messag.
 Transp - Transport - segment
 Netw - Network - routing (IP) → datagram
 Link - Link - frame
 Phys - Physical - bits

Network core:

Ch-1 Summary



message

$$C = B \log_2 (1 + S/N)$$

dist = $\frac{L}{R}$

packet in case, circuit switching, packet switching (statistical multiplexing)

@ Router
 ① Processing - ② Queuing - ③ Transmission
 N links $N \left(\frac{L}{R} \right)$
 P packets $(N-1) \frac{L}{R} + P \left(\frac{L}{R} \right)$

$$\frac{L}{R} \leq \frac{0.5}{1} \rightarrow \text{bottleneck}$$

layering merges it all
 Application (msg)
 Transport (segment, process-process)
 Network (datagram, host-host)
 Link (frame, link-to-link)
 Physical (bits)

Router

ISP - ISP
 core
 global
 transit
 edge
 IXP - peering
 buffering, spooling, etc

b4 security analysis
 routing (Google etc) → Dijkstra/BF
 Forwarding algorithm.

ch-2

Pipe
 end system code

Client-server

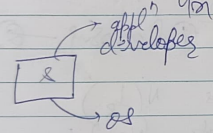
server: always on host
 permanent IP:
 client:

Peer to peer (P2P)

video call: every
 person: you off
 store
 400 bandwidth.

Self scalability

asymmetric
 socket



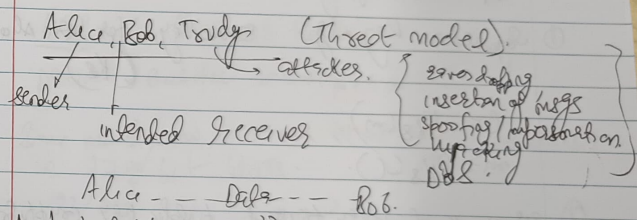
Zoom
 ↓
 services

7/9/06

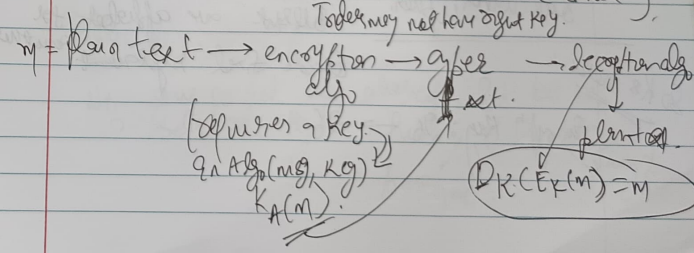
Ch-2

why we need Network Security →
 that if someone malicious is attempting to bypass original.

- (keep msg secret) ① Confidentiality: encrypt - decrypt. cipher text.
- (know who you are) ② Authentication: who actually sent it?
- (need integrity) ③ Integrity: sender but exact msg user.
- (use resources when needed) ④ Access & availability.



How to protect the msg??
 Principles of cryptography (send the msg but not be able to make sense).
 Trudy may not have right key.



Attacker of abilities

(Breaking encryption scheme)

- ① Brute force. Try every possible key.
(Try every key until it works)

② Statistical analysis

$\times Q \times r \times Q \times$

→ which chars occur most, patterns repeated sequences.

→ Plaintext attack / chosen-PTA

Types of Cryptography

- ① Symmetric key algo. Same key for both sender & receiver.
(K_s)

$$C = E_{K_s}(m)$$

$$m = D_{K_s}(C)$$

- ② as → third party trusted public key cryptography.
(key distribution / key exchange)

Substitution cipher

Block or alphabet for encryption
break fixed replacement.

So K_s here

"encryp" Key: 26 → 26

② DES

56 bit symmetric key: size fixed for algorithm (data)

→ 2^{56} keys

Plaintext: 64 bit

PT + Key → DES → ciphertext.

DES → 2 DES (more secure)
Back cipher (64 bits at a time)
(3 times encrypt w/ 2 diff keys)

③ AES (replacement for DES)

DES is Brute force become practical.

→ Same model now, same secret key, process data in 128 bit blocks.
192/256

So exponential growth in AES decryption.

Public key cryptography

No same secret key shared. But has key
2 selected keys (Public + Private)
to encrypt. → only to Bob

(2 + 2)

Bob publishes his ~~private~~ public key.
 Alice encrypts using Bob's public.
 any ~~body~~ Bob decrypts using private key.

→ (Public Encryption + Private Decryption)

Pub Key & Priv Key are mathematically related.

eg. $D_{\text{priv}}(E_{\text{pub}}(M)) = M$
 see that if Alice has Bob's 41275.

→ we can't reverse engineer private from public & we don't have that much time for it.

* RSA → modulus arithmetic using selected Public & Private.

$$K_B^{-1}(K_B(m)) = m.$$

Pub Key: (n, e) , $C = m^e \bmod n$: Encrypt
 Priv. Key: (n, d) , $m = C^d \bmod n$: Decrypt

$$\Rightarrow m = (m^e)^d \bmod n$$

~~the~~ order of public & private doesn't matter.

$$K_B^{-1}(K_B(m)) = m = K_B(K_B(m))$$

$$K_{\text{Public}}(m) = m^e \bmod n$$

$$K_{\text{Private}}(m) = m^d \bmod n.$$

$$\text{Pub} \rightarrow \text{Priv} \Rightarrow (m^e)^d = m^{ed}$$

$$\text{Priv} \rightarrow \text{Pub} = (m^d)^e = m^{de}$$

eg

message m (plaintext) → ciphertext

Public

$$K_A^+(m) = m^4$$

$$K_B^+(m) = m^4$$

$$K_C^+(m) = m^{11} \text{ (makes sense)}$$

C's public

So m was encrypted by C's private key

$$K_C^+(K_C^-(C))$$

So I is set by plaintext.

we're doing Authentication

→ for confidentiality, sender must use Recv's public for encryption

(Only knows how shared key)

Alice encrypts R using shared key

$$\text{Encrypted } R = E(K_{AB}, R)$$

and back to Bob

$$\text{Decrypt}(E(K_{AB}, R)) = R$$

So confirmed that it must be Alice.
Even if Tiny eavesdrops, it would be useless on new plaintext.

also →

4.0 drawback: key management tough.

So we try to solve authentication using public key techniques.

Setup:

Alice $\begin{cases} K_A^+ \\ K_A^- \end{cases}$ (fully aware)

Bob $\begin{cases} K_A^+ \\ K_A^- \end{cases}$

Actual

Alice → Bob. E → Alice.

To prove she was her private key (K_A^-)

So R = 9998 → encrypt using K_A^- → Bob uses K_A^+

Trusted Third Party

Public Key Certification Authorities (CA)

Bob & Alice at public key dist. center?

How to verify it actually belongs to Alice?
PK. Cryptography only says this msg corresponds to this key pair, but not who key is. Alice can't forge CA's digital signature.

So answer that Bob knows is really Alice's Public Key $\Rightarrow$ CA's certificate. It binds an identity to the public key.

4. Tony $\Rightarrow$ 1 order for Bob. Issues K_B

* Message Integrity

Long message (m) $\rightarrow$ Hash fun (H) $\rightarrow$ H(m) (fixed length)

as encryption expensive for long msgs.

Prob of hash.

$H \approx 1$

fix size

one way

Checksum $\Rightarrow$ diff msgs same hash

Challenge: Design an email, send system and public key system.
(Cryptography design problem)

$\rightarrow$ Authentication + Integrity + Confidentiality

Alice $\rightarrow$ Bob

Is this really good for me?

Is this message modified in?

Toolbox. $K_B, M \rightarrow H(M)$, CA certificates.

Active: Alice $\rightarrow$ Bob. Confirms 2 confs. $[M, H(M)]$

Bob: Tony on intercept & change $M \rightarrow M'$
 $H(M) \rightarrow H(M')$

Bob gets $[M', H(M)]$

So No authentication X

(MS)

$h = H(M)$ Alice.

Signature = Encrypt (also private key). $\{S = \text{sign}(K_B, h)\}$

So Alice sends $[M, S]$ to Bob.

Bob knows K_B

He uses it to verify

Signature & meanwhile also confirms

$h_1 = H(M)$

$h_1 = h_2$

and we're satisfied by whatever process
public key

BUT

Bob:

how do I know K_A is actually Alice.
 But need certificate.

Summary:

Alice sends $\left[\begin{array}{l} \text{Small } (m) \\ \text{Digital signature } (s) \text{ using } K_A \\ \text{Alice's certificate} \end{array} \right] \rightarrow \text{Bob.}$

Bob verifies CA public, gets K_A , finds
 hash himself $h_1 = H(m)$, $h_2 = \text{verify}(K_A, s)$
 $h_1 == h_2$, if yes — success.

Alice shares M + signature,

Bob does:

$h_1 = H(\text{received } m)$.
 $h_2 = \text{verify}(K_A, \text{signature})$.
 to check original hash.

If attacker comes

$M = \text{pay } \$100$, but I correspond to this
 $M' = \text{pay } \$1000$.

$H_1 = H(M)$ ← Bob does
 $H_2 = H(M')$

We have not yet solved confidentiality
 problem!

Exmpl: → authentication: $\text{Auth} \text{ of } s$?
 → integrity: mess age chng (2nd alert s^d)
 → confidentiality: if up d alert 2nd

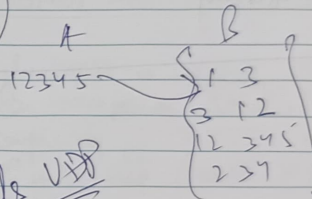
12/1/26

online class (Ch-3)

↓ cover up

- How Multiplexing works?
- Connections.
- T.C: 4 tuple.
- Telnet, letter

UDP



~~TCP vs UDP~~

order may not be order
 segments may be lost.
 UDP only sends the
 packet unlike TCP,
 no delay. centre for fax
 is to reach faster.

SNMP → transport w/in a small network itself
 → HTTP/3 → reliability ensured at application layer

RFC 768 breakdown.

UDP segment header

total src code = 16 bits = $2^4 \times 2^{10}$

$$n(n-1)/2 = 5$$

UDP Checksum

header → 32 bits
16 bit checksum
16 bit data → addition
for error

* Catch possibility of bit error
when checksum
Packet lost

↓
Packet breakdown

11/9/26

UDP Segment Header

Break data in 16 bit words
Data + checksum

↓
interval

receiver verifies everything
(to check if not changed during transmission)

Reliable Data Transfer (RDT) ⇒

- If network is unreliable then TCP & other protocols, how can they provide reliable communication
- complexity of RDT will depend on characteristics of unreliable channel. (loss, corrupt, reorder)

(Diagram needed) Reliable Data Transfer (RDT) (Perfect channel)
 Corrupt: check sum + feedback (ACK/NAK)
 Delay: timer + timeout + retransmission
 Reorder: sequence numbers + ordering log

→ sender & receiver don't know their state. So in distributed systems & machines maintain state of other

* use F8y to specify sender, receiver.

rdt2.0!

channel which may flip bits in packet.

10110110 $\rightarrow$ 10100110

sends ^{msg} receives

Some use checksum to detect.

Packet = [data | checksum]

Receiver?

Human analogy: feed back + retransmission.

$\rightarrow$ Please Resend! Packet down from

NACK: Negative Acknowledgment.

ACK: Acknowledgment.

Parity bit.

* channel doesn't guarantee, send & wait

Flow: what if ACK is corrupted.

$\rightarrow$ sender doesn't know ^{NACK} what happened at receiver.

$\rightarrow$ we sequence sequence numbers to avoid duplicate ordering

Packet A: [seq = 0 | data]

0/1 are enough. $\rightarrow$

Next: [seq = 1 | data]

sender can handle

Next: [seq = 0 | data]

only 1 packet at a time

Seq = 1 [data]

(stop & wait).

Protocol for Bit flip $\Rightarrow$ ACKNOWLEDGMENT $\neq$ Seq. Number

Receiver doesn't deliver duplicate pkt., already processed $\Rightarrow$ DISCARD.

rdt2.1.

Data = Hello.

Packet = [seq = 0 | data = hello | checksum]

(1) Rcvr got packet. Packet 0 $\checkmark$
(ACK sent back)

Now sender sends next packet seq = 1

(2) Rcvr got corrupted packet
checksum fail, NACK sent.

$\rightarrow$ so some packet is resent.

$\rightarrow$ so seq. number don't change ^{if} $\rightarrow$ if it is called packet it valid ACK ^{is} correct!

rdt2.2.

rdt2.1 problem: sender got ACK NACK corrupt.
 $\rightarrow$ to avoid duplicate

Soln $\rightarrow$ NACK start, if ACK use that

ACK 0: seq 0 correctly ^{from} NACK

ACK 1: seq 1 " "

$\leftarrow$ ACK 0 for tally ~~to~~ I have it.

CNTUT

(1) DNS

IP $\rightarrow$ not in local DNS cache $\therefore$
Non persistent HTTP lookup.

$$\text{DNS delay} = \sum_{i=1}^N RTT_i$$

TCP handshake $\rightarrow 1 RTT$

HTTP $\rightarrow 2 RTT$

Total = DNS time + 2 RTT (Non persistent)

$$T = \left(\sum_{i=1}^N RTT_i + 2 RTT_0 \right)$$

(2) HTTP references & additional objects
on same server
Total obj = 9

transmission for obj ≈ 0 & only RTT
ignore DNS.

(2.1) $9 \times 2 RTT_0 = 18 RTT_0$

(2.2) 6 Browsers 1161.

6 in one batch.
Per batch: $2 RTT_0$
 $2 RTT_0 + 4 RTT_i = 6 RTT_0$

(2.3) Non-persistent $\Rightarrow 2 \times 8 = 10 RTT_0$

(2.4) ~~Non-persistent~~

$$R = 15 \text{ Mbps} = 15 \times 10^6 \text{ bits/sec}$$

$$\Delta = 1 \text{ GB} = 10^9 \text{ bits}$$

$$b = 16 \text{ sec}$$

(2.5)

Distort = 3 sec.

$$D_{access} = \frac{\Delta}{1/\Delta b}, \quad \Delta = \frac{\text{obj size}}{\text{link rate}}$$

$$\Delta = \frac{10^9}{15 \times 10^6} = \frac{1}{15}$$

$$\Delta b = \frac{10}{15}$$

$D_{access} < 0$, $\Rightarrow \Delta b > 1 \Rightarrow$ handle faster than transmission

$\therefore$ given streams $\Rightarrow D_{access} \rightarrow \infty$
 $\Rightarrow$ Avg obj time $\rightarrow \infty$

(2.6) more obj = 0.9 $\Rightarrow$ 40.7. go to internet access link.

$$b = 0.4 \times 10^6 = 6.4 \text{ obj/sec}$$

(effective rate)

$$\Delta b' = \frac{6.4}{15} = 0.42$$

$$D_{access} = \frac{0.982}{1 - 0.42} \approx 0.1638$$

$$\begin{aligned} \text{hit} &= 0.6, \text{miss} = 0.4 \\ \text{Dhit} &= 0. \quad \text{Data cost + Data recd} \\ & \quad 0.1163 \text{ P3} \end{aligned}$$

$$\begin{aligned} \therefore \text{Davg} &= 0.6(0) + 0.4(2.1163) \\ &= \underline{0.84652} \end{aligned}$$

(14) Link length = 10m $\Rightarrow$ prop delay is negligible.

$$\begin{aligned} R &= 150 \text{ bit/sec} \\ \text{LP} &= 100000 \text{ b/h} \quad (\text{Data packet}) \end{aligned}$$

~~Given~~

$$L_c = 200 \text{ b/h} \quad (\text{Control packet ACK, TCP, HTTP})$$

parallel HTML obj
server as 10 more objects

$$\text{Total obj} = 11$$

Parallel TCP, $N=10$

$$TD = \frac{100000}{150} = \underline{666.67s}$$

$$TC = \frac{200}{150} = \underline{1.33s}$$

(15) Non Persistent, No 11th.

for each obj, TCP handshake

$$T_{\text{one obj}} = \underset{\substack{\uparrow \\ \text{control}}}{3T_c} + \underset{\substack{\uparrow \\ \text{HTTP}}}{T_c} + \underset{\substack{\uparrow \\ \text{obj}}}{TD} = \underline{672s}$$

$$T_{11} = 11 \times T_{\text{one}} = \underline{7392s}$$

sub) Non Persistent + 10th TCP.

~~Given~~ HTML arrived $\Rightarrow$ 672s

10 additional now,

$$\therefore 10 \text{ TCP conn, each obj} = \frac{150}{10} = 15 \text{ b/h}$$

$$T_c = \frac{200}{15} = 13.33$$

$$\text{Data packet} = \frac{100000}{15} = \underline{666.67s}$$

$$\text{for each conn, } 4 \times 13.33 + 666.67 = \underline{672s}$$

$\therefore$ all 10 in 11th

$$\therefore T_{11}^{\text{total}} = 672 + 6720 = \underline{7392s}$$

SAMS

→ By sense, Request TCP handshake

② Resend HTTP No Pipelining

↓ TCP & sense for 11 obj

Of: $3T_c + T_c + T_D = 672$
for send 10 → no new TCP is reqd.
Each needs.

$$HTTP + data = T_c + T_D = 6688$$

$$for 10 = 668 \times 10$$

$$Total = 672 + 6680 = 7352$$

$$\Delta = 408$$

→ Req 01 → resp 01, ~~02~~

resp 02 → resp 02

in the system w/ Pipelining we send
reqs w/o waiting.

③

~~1 page~~
web page → 6 things: (1 video 5 images).

Video → 2000 frames
(VVVVVVV...)

1 image → 3 frames
AAA
BBB
...

send V → 1 FTT

⑤1. VVV...V → images

⑤2. VABCDE - VABCDE - - -

$$2000 \times 15 = 30000$$

(600)
V I I I I I I

R₁ → 6700 frames

R₂ → 6

after 3 Reads, I₁ = 3 - -

$$\therefore \text{Total frames for images} = 3 \times 6 \text{ frames} = 18 \text{ frames}$$

$$\text{⑥ } T_{\text{req}} \approx T_{\text{res}}/B_s > V$$